

CSCE 713 SOFTWARE SECURITY · MODULE 06 · LECTURE NOTES

Deserialisation and Trust Boundaries

Reconstructing objects from hostile bytes.

MODULE OUTCOMES

- Explain why native deserialisation is dangerous.
- Explain gadget chains conceptually.
- Explain the safe alternatives.
- Explain SSRF and the confused deputy at this layer.
- Audit a trust boundary crossing.

1 Why native deserialisation is dangerous

NATIVE DESERIALISATION LETS THE INPUT CHOOSE WHICH TYPES TO CONSTRUCT AND WHICH METHODS TO RUN

A **native serialisation format encodes the class as well as the data** — that is what makes it convenient — so **the input decides what gets instantiated**, which is an extraordinary amount of authority to hand to a byte stream from outside your system.

And construction runs code. Constructors, property setters, validation hooks, custom readObject methods, and finalisers all execute during deserialisation — before your code has seen the resulting object or had any opportunity to inspect it, which means no amount of post-deserialisation validation helps.

So the attacker's problem reduces to finding a reachable class whose construction or disposal does something useful — and in a large dependency tree, such classes reliably exist (§1's gadget discussion below).

Which is why the rule is categorical rather than conditional: never deserialise untrusted data with a format that encodes types — Java's native serialisation, Python's pickle, PHP's unserialize, .NET's BinaryFormatter, Ruby's Marshal. **Several of these are now deprecated by their own platforms for exactly this reason**, which is unusual and tells you how bad it is.

- A **'gadget'** is a class already present in your application or its dependencies whose deserialisation has a **side effect** — it writes a file, opens a network connection, invokes a method on another object it holds, or triggers a lookup.
- A **chain composes several of them**, so that the side effects compose into something the attacker wants — typically arbitrary code execution, reached through four or five entirely ordinary classes.

- **And the attacker needs no vulnerability in your code at all. The gadgets are ordinary, correct classes behaving exactly as designed**, assembled in an order and a combination their authors never considered and had no reason to.
- **Which is precisely why the dependency tree matters here — adding a library adds gadgets**, and the gadget surface grows with the transitive closure (CSCE 701 Module 08 §1's trust argument, arriving with a specific consequence).
- **And why an allowlist of permitted classes is the only mitigation that works** if you cannot change the format — **a blocklist of known gadget classes is always incomplete**, because new chains are found in existing libraries regularly. **The key insight is that no component is buggy**, which is why **this defect is invisible to any analysis that examines components individually** — and is Module 01 §1's composition cause in its purest form.

2 The alternatives

```
BEST: A DATA-ONLY FORMAT, PARSED INTO A KNOWN TYPE
      JSON, Protocol Buffers, MessagePack, CBOR --
      formats that encode DATA and not TYPES.
      Then parse into a type you named in your own code
      (Module 04 section 4's discipline).
```

```
ALSO FINE: A SCHEMA
      Protobuf or Avro with a declared schema. The shape
      is fixed by your code rather than by the input.
```

```
IF YOU CANNOT CHANGE THE FORMAT
      an allowlist of permitted classes, enforced during
      deserialisation -- NOT a blocklist of known
      gadgets, which is always incomplete
```

```
AND IN EVERY CASE
      authenticate the data before deserialising it, if
      it crossed a boundary you control both ends of
      (CSCE 711 Module 04) -- a MAC makes the question
      of trust actually answerable
```

'Encodes data, not types' is the test, and it immediately classifies every serialisation format you are likely to encounter — which makes it the portable version of the rule, usable in a language you have never written. **Note that a data-only format can still be misused**: a JSON document whose contents name a class your code then looks up and instantiates has reintroduced the whole problem, and this pattern appears in several real frameworks' polymorphic deserialisation features.

AND A MAC TURNS UNTRUSTED DATA INTO DATA YOU CAN REASON ABOUT

If you serialised it and you are the one who will deserialise it, authenticate it — a MAC computed over the serialised bytes, **verified before any parsing occurs** (CSCE 711 Module 04 §4's ordering rule).

Which converts 'untrusted input' into 'input we produced' — and that is a genuinely different trust category rather than a mitigation applied to the same category. The question 'can an attacker influence this' now has the answer 'no'.

This is exactly why signed cookies and signed tokens work, and why the signature must be verified before the payload is parsed rather than after — a token whose claims are read first and verified second has already acted on attacker-controlled data.

But it does not help at all where the data genuinely originates elsewhere — a third-party API, a user upload, a message from another organisation — **where §2's format choice is the only available answer**, and where the temptation to use the convenient native format is strongest.

3 Server-side request forgery

SERVER-SIDE REQUEST FORGERY: YOUR SERVER FETCHES A URL THE USER CHOSE

The request originates inside your network, from a host holding your server's network position and its credentials — so it reaches internal services that are deliberately not exposed externally, and those services frequently have weak or absent authentication precisely because they are internal (CSCE 701 Module 04 §2).

Which is the confused deputy pattern (CSCE 701 Module 03 §2): the server acts on its own authority on behalf of a requester who does not hold that authority — the identical structure as CSRF, path traversal, and missing object checks, at the network layer.

And the cloud metadata endpoint makes it considerably worse — a link-local address that returns credentials to any process on the host that asks for them, which turns a URL-fetching feature into credential theft and then into whatever those credentials permit.

So the defences are: an allowlist of permitted destinations, resolution of the hostname followed by validation of the *resolved address* rather than the name, refusing to follow redirects, and egress filtering at the network level (CSCE 701 Module 04 §2) — and the egress filtering is what limits the damage when the application-level checks are bypassed.

- Validating the hostname rather than the resolved address — DNS can return an internal address for a name you permitted, and an attacker who controls a domain controls what it resolves to.
- And DNS rebinding: the name resolves to a permitted external address when you check it and to an internal address when you connect — which is Module 05 §3's TOCTTOU arriving at the network layer, and is the pitfall that makes address-checking insufficient on its own.
- Following redirects, which lets the first response choose the second destination — so a permitted URL returns a 302 to the metadata endpoint, and your validation never saw it.
- Alternative URL schemes — file:, gopher:, dict: and others, which reach things HTTP does not and which a URL parser may accept while your validation only considered HTTP.
- And IPv6, decimal, octal, and mixed address encodings, which defeat textual filters looking for 127.0.0.1 or 169.254. The robust answer is to resolve the name once, validate the resulting address, and then connect to that address directly rather than to the name — which closes the rebinding window and is Module 05 §3's operate-on-the-handle principle again.

4 Auditing a trust boundary

- What format is the data in, and does that format encode types or behaviour rather than only data? (§2's test.)
- Is it authenticated, and is the authentication verified *before* parsing rather than after? (§2's callout.)
- What type does it become, and was that type named by my code or chosen by the input? — which is Module 04 §4's parse-don't-validate question asked at a boundary.

- **What does the parsing itself do** — allocate based on a declared length (Module 03 §1), decompress (which is where decompression bombs live), construct objects, resolve external references, or follow links?
- **And whose authority is used to act on it** — the requester's, or the process's? (§3's confused deputy.) **The last question generalises this entire module: deserialisation and SSRF are both the same error — acting with your own authority on a requester's instruction**, which is why the fix in both cases is to constrain what the instruction may ask for rather than to filter the instruction.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Frohoff & Lawrence — Marshalling Pickles (free)	§1's gadget chains in the original — the talk that made this class widely understood.
OWASP — Deserialization and SSRF cheat sheets (free)	§2 and §3 as current, per-platform guidance , maintained and specific.
Google — the Java serialization deprecation rationale (free)	§1's severity, as judged by the platform's own maintainers — a language feature deprecated for security reasons, which is rare.
CWE-502, CWE-918 (free)	§1 and §3 with the shared vocabulary .

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Deserialise a pickle you crafted** in your own sandbox and observe code running.
- 2 **Explain why post-deserialisation validation cannot help.**
- 3 **Find three classes in your dependency tree** whose construction has a side effect.
- 4 **Convert one native serialisation use to JSON** parsed into a declared type.
- 5 **Add a MAC to a serialised value** and verify it before parsing.
- 6 **Show what happens if you verify after parsing instead.**
- 7 **Write a URL-fetching endpoint** and then attack it from your own client with an internal address.
- 8 **Implement resolve-then-validate-then-connect** and confirm it blocks rebinding.
- 9 **Test your filter against decimal and IPv6 encodings** of a loopback address.
- 10 **Run Part 4's five questions** at one real boundary in your own system.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is native deserialisation dangerous — give both mechanisms?
- 2 Why does post-deserialisation validation not help?
- 3 Name five formats to avoid for untrusted data.
- 4 What is a gadget chain, and why is no component buggy?
- 5 State the format test and name three safe formats.
- 6 What does a MAC change, and when does it not help?
- 7 Why is SSRF severe, and what pattern is it?
- 8 Give four SSRF defences and five pitfalls.
- 9 Why is DNS rebinding the hardest pitfall?
- 10 Give the five boundary questions and say which generalises the module.